



ELSEVIER

Contents lists available at ScienceDirect

Information Systems

journal homepage: www.elsevier.com/locate/infosys



The wavelet matrix: An efficient wavelet tree for large alphabets[☆]

Francisco Claude^{a,1}, Gonzalo Navarro^{b,*}, Alberto Ordóñez^{c,3}

^a Esc. Inf. & Tel., Univ. Diego Portales, Chile

^b Department of Computer Science, University of Chile, Chile

^c Database Laboratory, Univ. da Coruña, Spain



An Efficient Wavelet Tree for Large Alphabets

Claude et al. (2015)

Slides created with the assistance of Claude Sonnet (a *different* Claude)

Motivation: Why Do We Need This?

The Problem

Genomic sequences are over large alphabets (e.g., amino acids, k-mers)

Wavelet trees are complex to implement efficiently for large σ

Pointer-based trees waste space & hurt cache performance

We need fast rank/select/access — the backbone of compressed FM-indexes



The Wavelet Matrix Solution

Flat array layout (no pointers) → cache friendly

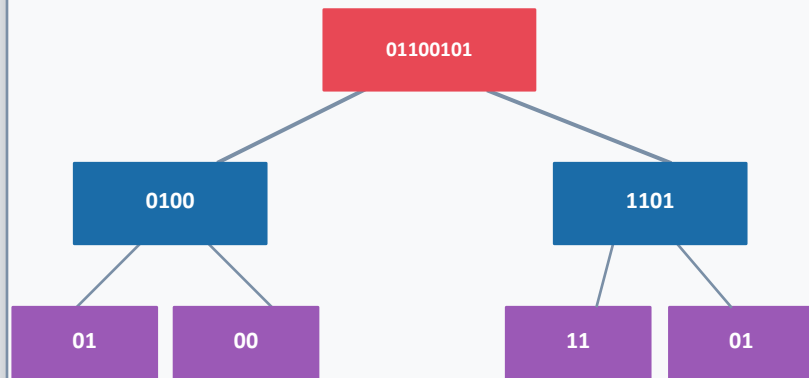
Identical to wavelet tree in bits — same theoretical bounds

Simpler navigation via bit-rank on each level

$O(\log \sigma)$ time for access, rank, select

Wavelet Tree vs. Wavelet Matrix

Wavelet Tree



Each level stored in a SEPARATE node
Pointer overhead per node
Navigation follows tree branches

VS

Wavelet Matrix

L_0	0	0	1	0	1	1	0	1
L_1	0	1	0	0	1	1	0	1
L_2	1	0	1	1	0	0	1	0

All levels stored as FLAT BIT ARRAYS
No pointers, no tree nodes
Navigation via bit-rank only

Our Running Example

We will use this sequence throughout:

index:	0	1	2	3	4	5	6	7
S =	3	1	4	1	5	9	2	6

Alphabet: $\Sigma = \{1,2,3,4,5,6,9\}$ ($\sigma = 7$ distinct values)

Bits per level: n

Number of levels: $\lceil \log_2 \sigma \rceil + 1 = \lceil \log_2 7 \rceil + 1 = 4$ levels

Binary representations (4 bits):

1 $\rightarrow$ 0001 2 $\rightarrow$ 0010 3 $\rightarrow$ 0011 4 $\rightarrow$ 0100

5 $\rightarrow$ 0101 6 $\rightarrow$ 0110 9 $\rightarrow$ 1001

We will answer:

access(5)

What is $S[5]$?

rank(3, 4)

How many 3s in $S[0..4]$?

select(1, 1)

Where is the 2nd occurrence of 1?

Construction: Building the Wavelet Matrix

$S = [3, 1, 4, 1, 5, 9, 2, 6]$ (4 levels for $\sigma \leq 16$) ← sequence

Level 0 (bit 3 = MSB)

3	1	4	1	5	9	2	6
0	0	0	0	0	1	0	0

Level 1 (bit 2)

3	1	4	1	5	2	6	9
0	0	1	0	1	0	1	0

Level 2 (bit 1)

3	1	1	2	9	4	5	6
1	0	0	1	0	0	0	1

Level 3 (bit 0 = LSB)

1	1	9	4	5	3	2	6
1	1	1	0	1	1	0	0

z=7	3	1	4	1	5	2	6	9
-----	---	---	---	---	---	---	---	---

z=5	3	1	1	2	9	4	5	6
-----	---	---	---	---	---	---	---	---

z=5	1	1	9	4	5	3	2	6
-----	---	---	---	---	---	---	---	---

done

← z val

← rearranged numbers

The Final Wavelet Matrix Structure

Four bit arrays + four z-values (count of 0s per level)

index:	0	1	2	3	4	5	6	7	z values →
L_0	0	0	0	0	0	1	0	0	$z=7$
L_1	0	0	1	0	1	0	1	0	$z=5$
L_2	1	0	0	1	0	0	0	1	$z=5$
L_3	1	1	1	0	1	1	0	0	$z=3$

Each level stores a rank_1 / rank_0 structure for $O(1)$ queries. Space: $n \lceil \log_2 \sigma \rceil$ bits total.

Operation: access(i) — What is $S[i]$?

Algorithm: access(i)

1. Start with position $p = i$
2. For each level $\ell = 0 \dots \log_2 \sigma - 1$:
 - a. $\text{bit} \leftarrow L_\ell[p]$
 - b. if $\text{bit} == 0$:
 $p \leftarrow \text{rank}_0(L_\ell, p)$
 - c. else:
 $p \leftarrow z_\ell + \text{rank}_1(L_\ell, p)$
3. Decode final $p \rightarrow \text{symbol}$

Key Intuition

Each level partitions symbols into 0-half and 1-half. The z value marks the boundary. We follow a position from level to level, tracking which half it falls into — this re-assembles the original symbol bit by bit.

Time: $O(\log \sigma)$ — one bit-rank per level

Space: included in level's rank structure

z_ℓ = number of 0s in level ℓ
= boundary between 0-children and 1-children

access(5) — Step-by-Step Example

Query: $\text{access}(5) \rightarrow$ What is $S[5]$? (Expected answer: 9)



Result: Bits 1001 = 9 in decimal $\rightarrow S[5] = 9$ ✓

Operation: $\text{rank}(c, i)$ — How Many c 's up to i ?

$\text{rank}(c, i)$ = number of occurrences of symbol c in $S[0..i]$

Algorithm: $\text{rank}(c, i)$

1. $p \leftarrow i$ (position pointer)
2. For each level $\ell = 0 \dots \log_2 \sigma - 1$:
Let $b \leftarrow$ bit ℓ of c (MSB first)
if $b == 0$:
 $p \leftarrow \text{rank}_0(L_\ell, p)$
else:
 $p \leftarrow z_\ell + \text{rank}_1(L_\ell, p)$
3. $\text{rank}(c, i) = p - (\text{start of } c\text{'s range}) + 1$

Intuition

We trace symbol c bit-by-bit from level 0 down. At each level, we follow the same path that c would follow during construction. The final position p tells us where c 's occurrences have been mapped.

Symbol Range after Construction

After all levels, identical symbols are grouped contiguously. The range $[sp, ep]$ of symbol c can be computed in $O(\log \sigma)$ time. The answer is:

$$\text{rank}(c, i) = p - sp + 1$$

rank(1, 4) — How Many 1s in S[0..4]?

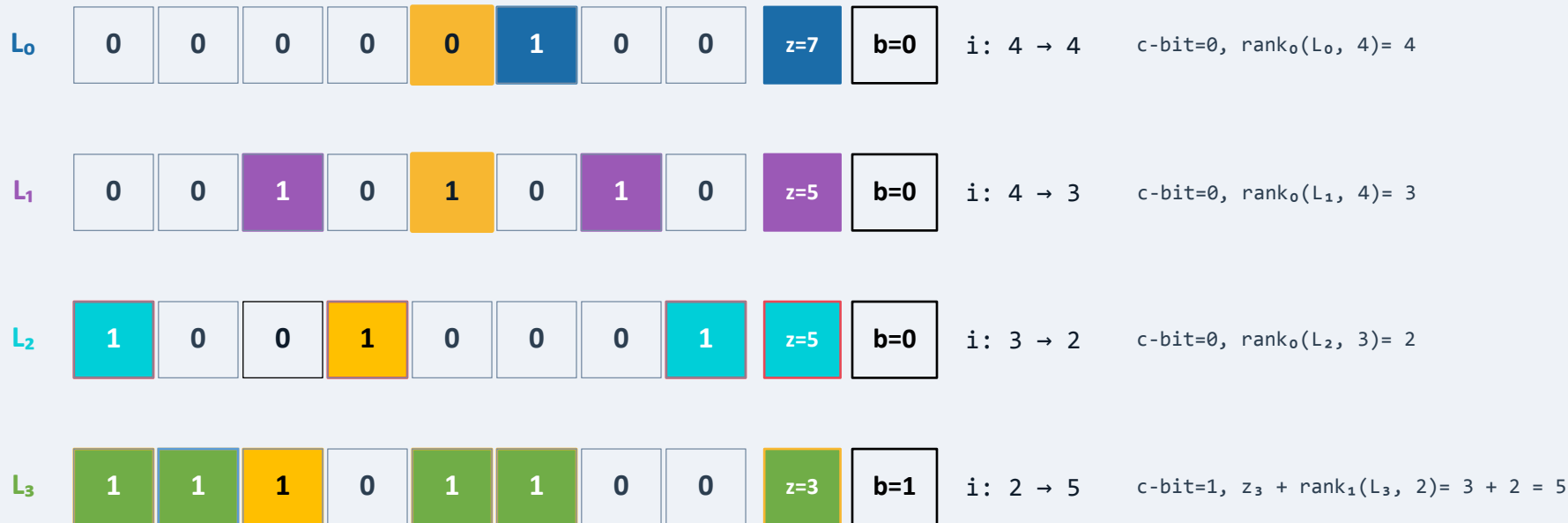
$c = 1 = 0001$ in binary | $i = 4$ | Expected: 2 ($S = [3, 1, 4, 1, 5, 9, 2, 6]$)

$C[a]$ = starting position for occurrences of symbol 'a' in the *last level* of the wavelet matrix

$c = 1 =$

0	0	0	1
---	---	---	---

 Rank of 1 in S here



Final $i = 5$, $C[1] = 3 \rightarrow \text{rank}(1, 4) = 5 - 3 = 2$ ✓ (1 appears at S[1] and S[3])

Operation: $\text{select}(c, j)$ — Where is the j -th c ?

$\text{select}(c, j)$ = position of the j -th occurrence of symbol c in S

Algorithm: $\text{select}(a, j)$

1. $j \leftarrow C[a] + j$ (j -th pos in a 's range)
2. For each level $\ell = \log_2 \sigma$ DOWN to 0:
Let $b \leftarrow$ bit ℓ of a
if $b == 0$:
 $j \leftarrow \text{select}_0(L_\ell, j)$
else:
 $j \leftarrow \text{select}_1(L_\ell, j - z_\ell)$
3. Return j

Key Contrast with rank

rank traverses TOP-DOWN, following bits of c .

select traverses BOTTOM-UP, inverting the mapping layer by layer.

Time: $O(\log \sigma)$ — one bit-select per level

$\text{select}_b(a, k)$ = position of k -th b in a

Intuition: Working Backwards

We know where c lives in the final sorted array. Each $\text{select}_{1/0}$ call "unshuffles" one level of the construction, recovering the original position.

select(1, 1) — Where is the last index with 1 rank of 1?

$S = [3, 1, 4, 1, 5, 9, 2, 6]$

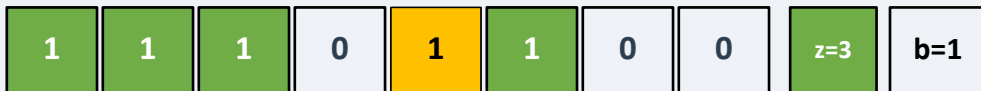
should find this index (3)

Step 1: $C[1] = 3$, so $j = 3 + 1$

Step 2: Traverse BOTTOM-UP ($\ell = 3 \rightarrow 0$) using select:

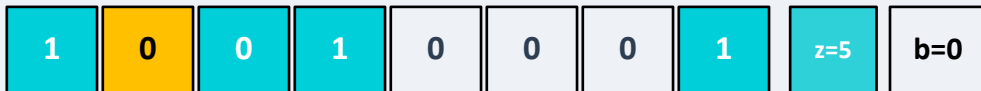
$c = 0001$ (bits at each level: $b_3=1, b_2=0, b_1=0, b_0=0$, reading LSB $\rightarrow$ MSB)

L3



$j: 4 \rightarrow 1$ $b=1, \text{select}_1(L_3, 4 - 3) = \text{select}_1(L_3, 1) = 1$

L2



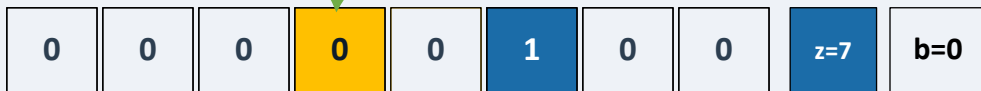
$j: 1 \rightarrow 2$ $b=0, \text{select}_0(L_2, 1)=2$

L1



$j: 2 \rightarrow 3$ $b=0, \text{select}_0(L_1, 2)=3$

L0



$j: 3 \rightarrow 3$ $b=0, \text{select}_0(L_0, 3)=3$

Result: $j = 4 \rightarrow \text{select}(1, 1) = 4$ ✓ ($S[3] = 1$, the 2nd occurrence of 1)

Wavelet Matrix vs. Wavelet Tree: A Comparison

Property	Wavelet Tree	Wavelet Matrix	
Memory layout	Pointer-based tree nodes	Flat bit arrays per level	WM wins
Cache efficiency	Poor (pointer chasing)	Excellent (sequential access)	WM wins
Implementation	Complex node management	Simpler — bit-rank only	WM wins
Space (bits)	$n \lceil \log_2 \sigma \rceil + o(n)$ overhead	$n \lceil \log_2 \sigma \rceil + \text{rank \& select}$	Same
$\text{access}(i)$	$O(\log \sigma)$	$O(\log \sigma)$	Same
$\text{rank}(c, i)$	$O(\log \sigma)$	$O(\log \sigma)$	Same
$\text{select}(c, j)$	$O(\log \sigma)$	$O(\log \sigma)$	Same
Practical speedup	—	2–3× faster in practice	WM wins

Key Takeaways

- 1 The wavelet matrix stores the same bits as a wavelet tree — but in flat arrays, level by level.
- 2 A single z-value per level (count of 0s) replaces all the tree pointers needed for navigation.
- 3 access, rank, and select all run in $O(\log \sigma)$ time using only bit-rank and bit-select primitives.
- 4 The cache-friendly layout yields 2–3× speedups over pointer-based wavelet trees in practice.
- 5 Wavelet matrices are foundational in modern compressed genomic indices (FM-index, GBWT, etc.).